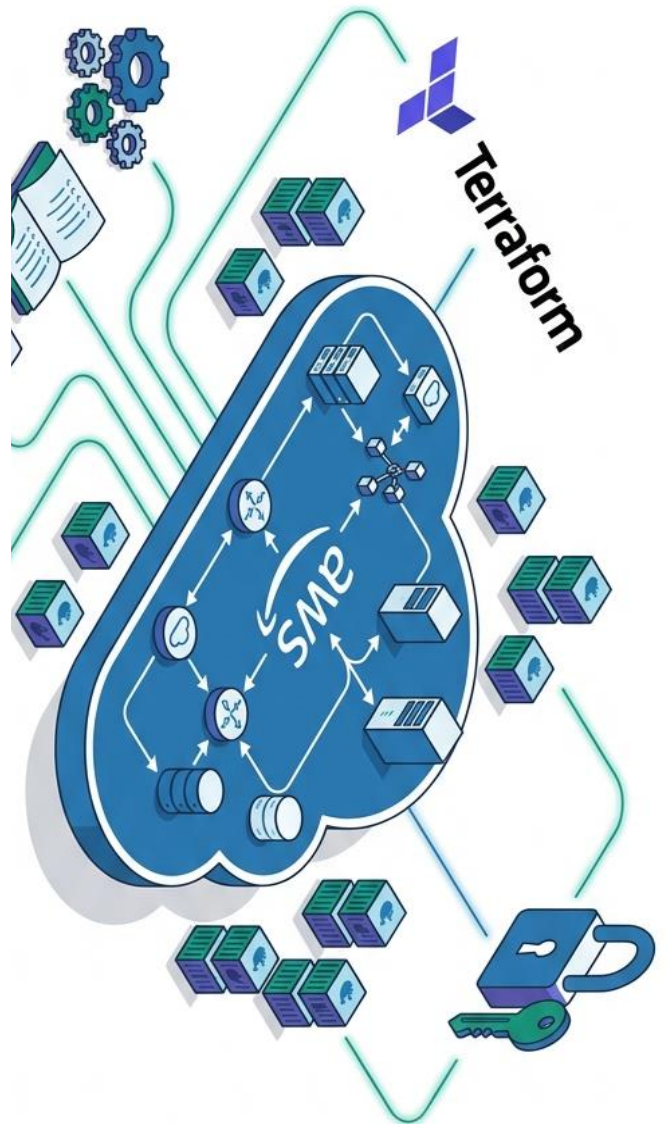


ACTIVIDAD INTEGRADORA

Iac y Seguridad

Learning Management System (LMS)



[01 MAYO 2026]

Integrantes del equipo:

Evelyn Betzabeth Sotelo Pichardo 3103108

Joshua Reyes León 5054355

Estrella Ximena Zarate Delgado 3101146

[Repositorio de GitHub](#)



1. Introducción y Contexto del Proyecto

Objetivo del Proyecto

El presente proyecto tiene como objetivo implementar la infraestructura completa para un Learning Management System (LMS) utilizando las mejores prácticas de DevOps, específicamente Infraestructura como Código (IaC), gestión automatizada de configuración y robustecimiento de seguridad.

Descripción del Sistema

El LMS es una plataforma educativa que requiere una infraestructura escalable, segura y fácil de mantener. El sistema debe soportar:

1. Múltiples usuarios concurrentes (estudiantes y profesores)
2. Almacenamiento de contenido multimedia (videos, documentos, imágenes)
3. Alta disponibilidad mediante balanceo de carga
4. Gestión segura de credenciales y secretos

2. Análisis de Requerimientos

Requerimientos Funcionales

ID	Requerimiento	Prioridad	Descripción
RF-01	Escalabilidad	Alta	El sistema debe soportar incrementos en la carga de usuarios mediante balanceo de carga
RF-02	Almacenamiento Persistente	Alta	Los archivos multimedia deben almacenarse de forma segura y accesible
RF-03	Alta Disponibilidad	Media	El sistema debe estar disponible 99% del tiempo mediante infraestructura redundante
RF-04	Gestión de Secretos	Alta	Las credenciales no deben exponerse en código fuente

Requerimientos de Infraestructura

ID	Requerimiento	Prioridad	Descripción
RI-01	Red Aislada (VPC)	Alta	Infraestructura desplegada en VPC privada con subredes públicas
RI-02	Seguridad (Security Groups)	Alta	Reglas de firewall para controlar tráfico SSH (22) y HTTP (80)
RI-03	Servidor de Aplicación (EC2)	Alta	Instancia t2.micro con Ubuntu 22.04 LTS
RI-04	Almacenamiento en la Nube (S3)	Media	Bucket S3 para archivos estáticos y backups
RI-05	Balanceador de Carga (ALB)	Alta	Application Load Balancer para distribuir tráfico

Dimensionamiento de Recursos

Basándose en los requerimientos, se determinó lo siguiente:

Recurso	Especificación	Justificación
VPC	CIDR: 10.0.0.0/16	Red privada con 65,536 IPs disponibles
Subredes	2 públicas (10.0.1.0/24, 10.0.2.0/24)	Redundancia en múltiples AZs
EC2	t2.micro (1 vCPU, 1 GB RAM)	Suficiente para entorno de desarrollo/pruebas
ALB	Application Load Balancer	Distribución de tráfico HTTP/HTTPS
S3	Bucket estándar	Almacenamiento de objetos estáticos
Security Group	SSH (22), HTTP (80)	Acceso controlado a servicios

Complejidad y dependencias de los requerimientos

ID	Requerimiento	Complejidad	Dependencias principales	Justificación
RF-01	Escalabilidad	Media	Application Load Balancer, EC2, subredes públicas	Permite aumentar la capacidad del sistema agregando más instancias EC2 detrás del balanceador de carga.
RF-02	Almacenamiento persistente	Media	S3 Bucket, permisos de acceso, aplicación LMS	Los archivos multimedia del LMS, como documentos, imágenes y videos, se almacenan en S3 para mantenerlos disponibles y separados del servidor.
RF-03	Alta disponibilidad	Media-Alta	Dos zonas de disponibilidad, subredes públicas, ALB	La infraestructura se prepara para trabajar en más de una zona de disponibilidad. En producción se podrían agregar más instancias EC2 para mejorar la tolerancia a fallos.
RF-04	Gestión de secretos	Alta	HashiCorp Vault, Terraform, Ansible, tokens de acceso	Las credenciales deben protegerse y no escribirse directamente en el código. Vault centraliza y controla el acceso a los secretos.
RI-01	Red aislada VPC	Media	VPC, subredes, Route Table, Internet Gateway	La VPC organiza la red principal del proyecto y permite controlar la comunicación entre los recursos.
RI-02	Seguridad con Security Groups	Media	EC2, reglas SSH/HTTP, VPC	Los Security Groups limitan el tráfico permitido hacia la instancia, reduciendo accesos no autorizados.
RI-03	Servidor de aplicación EC2	Media	AMI Ubuntu, subred pública, Security Group, Key Pair	La instancia EC2 funciona como servidor base donde se configura el entorno de la aplicación con Ansible.
RI-04	Almacenamiento en S3	Baja-Media	Bucket S3, política de acceso, aplicación LMS	S3 permite guardar archivos estáticos y respaldos de forma escalable, sin depender del almacenamiento local de la EC2.
RI-05	Balanceador de carga ALB	Media-Alta	Subredes públicas, Security Group, EC2 target	El ALB distribuye el tráfico web y deja preparada la arquitectura para escalar a más servidores.
RI-06	Configuración automatizada con Ansible	Media	Inventario, llave SSH, EC2, playbook site.yml	Ansible automatiza la instalación de Docker, dependencias y configuración inicial del servidor.

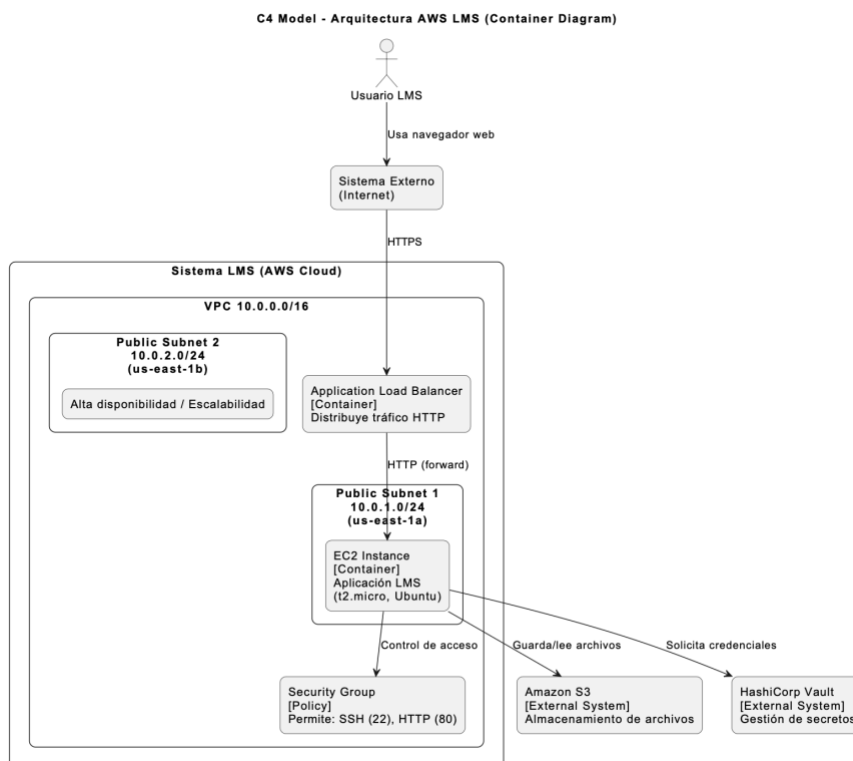
RI-07	Integración de Vault con Terraform/Ansible	Alta	Vault, secretos secret/aws, secret/database, Terraform, Ansible	Permite que las herramientas DevOps obtengan credenciales desde Vault sin exponerlas en archivos del repositorio.
-------	--	------	---	---

Con este análisis se observa que los componentes con mayor complejidad son la alta disponibilidad, el balanceador de carga y la gestión de secretos, ya que dependen de varios servicios trabajando en conjunto. Por otro lado, elementos como S3 o los Security Groups son más directos, aunque siguen siendo importantes para el funcionamiento y la seguridad del sistema.

3. Arquitectura de la Infraestructura

La infraestructura se organiza en tres partes principales para separar funciones y mejorar la seguridad:

- Capa de Red (VPC): Se usa una VPC con rango 10.0.0.0/16 que contiene dos subredes públicas en distintas zonas (us-east-1a y us-east-1b), lo que ayuda a mantener la disponibilidad del sistema.
- Capa de Aplicación (EC2 + Docker): La aplicación corre en una instancia EC2 con Ubuntu 22.04, utilizando Docker y Docker Compose para facilitar el despliegue y manejo de los servicios.
- Capa de Seguridad (Vault + Security Groups): Los Security Groups controlan el acceso permitiendo solo SSH y HTTP, mientras que Vault se encarga de guardar las credenciales de forma segura sin exponerlas en el código.



Descripción: El diagrama muestra la arquitectura del LMS en AWS, donde todos los recursos están dentro de una VPC (10.0.0.0/16), lo que permite mantenerlos en una red privada.

Se utilizan dos subredes públicas en distintas zonas (us-east-1a y us-east-1b) para mejorar la disponibilidad. En una se encuentra la instancia EC2 (t2.micro con Ubuntu 22.04) que ejecuta la aplicación, protegida por un Security Group que solo permite SSH y HTTP. En la otra se ubica el Application Load Balancer, encargado de distribuir el tráfico.

El acceso inicia desde internet a través del Internet Gateway, pasa por el Load Balancer y llega a la EC2. Los archivos se almacenan en S3.

En general, esta arquitectura permite tener mayor disponibilidad, seguridad básica y la posibilidad de escalar el sistema fácilmente.

Componentes de la Arquitectura

Componentes de Red

- VPC: 10.0.0.0/16 (lms-vpc)
- Subnet Pública 1: 10.0.1.0/24 en us-east-1a
- Subnet Pública 2: 10.0.2.0/24 en us-east-1b
- Internet Gateway: Conectividad a internet (lms-igw)
- Route Table: Tabla de rutas hacia 0.0.0.0/0 vía IGW

Componentes de Cómputo

- EC2 Instance: t2.micro con Ubuntu 22.04 (lms-server)
- Application Load Balancer: Distribuye tráfico HTTP entre instancias (lms-alb)

Componentes de Almacenamiento

- S3 Bucket: Almacenamiento de objetos estáticos (lms-bucket-joshua-548161)

Componentes de Seguridad

- Security Group (lms-web-sg):
 - Inbound: SSH (22) desde 0.0.0.0/0
 - Inbound: HTTP (80) desde 0.0.0.0/0
 - Outbound: All traffic hacia 0.0.0.0/0
- Key Pair: lms-key.pem para acceso SSH

1. Aprovisionamiento con Terraform (AWS)

Se creara la siguiente arquitectura en AWS con Terraform:

- VPC
- Subred pública
- Internet Gateway
- Tabla de rutas
- Security Group (SSH + HTTP)
- EC2 (servidor base para Ansible)
- S3 (almacenamiento)
- Load Balancer (ALB básico)

1. Estructura del proyecto

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx: ~/Documentos/iac/lms-iac/terraform
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac$ mkdir lms-iac
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac$ cd lms-iac
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac$ mkdir terraform
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac$ cd terraform
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

2. Instalación de Terraform (Ubuntu)

Como ya lo tengo instalado uso terraform version para verificar

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ terraform version
Terraform v1.14.9
on linux_amd64

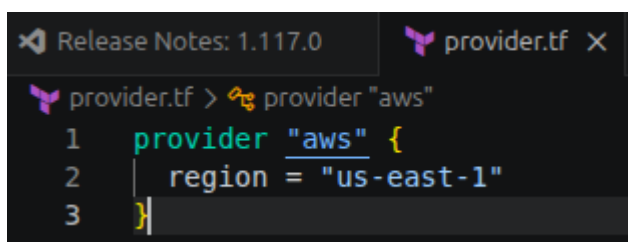
Your version of Terraform is out of date! The latest version
is 1.15.0. You can update by downloading from https://developer.hashicorp.com/terraform/install
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

3. Configurar AWS CLI

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ aws configure
AWS Access Key ID [*****J7S5]: AKIA7TVY5HDXOJG4L73
AWS Secret Access Key [*****XXhk]: PbUEroYbQLG1CwLQUE8nidVvqUag34D/MNPYxR8e
Default region name [us-east-1]: us-east-1
Default output format [json]: json
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

Para que Terraform pueda crear recursos en AWS, se configuró el AWS CLI con las credenciales de acceso

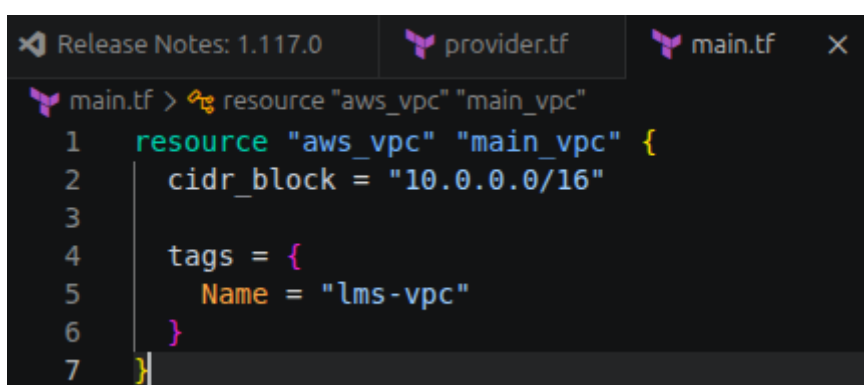
4. provider.tf



```
Release Notes: 1.117.0 provider.tf X
provider.tf > provider "aws"
1 provider "aws" {
2   region = "us-east-1"
3 }
```

- provider "aws": Indica que se usará AWS como proveedor de nube
- region = "us-east-1": Define la región de AWS donde se crearán los recursos (Virginia del Norte)

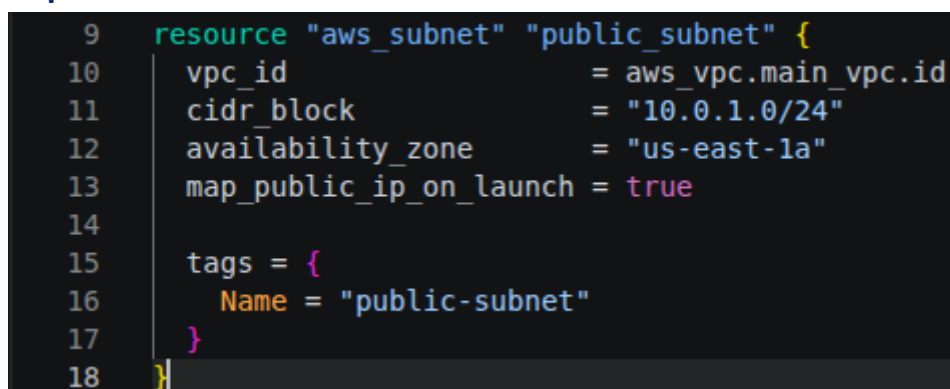
5. Crear VPC (main.tf)



```
Release Notes: 1.117.0 provider.tf main.tf X
main.tf > resource "aws_vpc" "main_vpc"
1 resource "aws_vpc" "main_vpc" {
2   cidr_block = "10.0.0.0/16"
3
4   tags = {
5     Name = "lms-vpc"
6   }
7 }
```

- Se crea una red privada virtual con rango de IPs 10.0.0.0/16
- Esto permite tener hasta 65,536 direcciones IP disponibles
- El tag "Name" facilita identificar el recurso en la consola de AWS

6. Subred pública



```
9 resource "aws_subnet" "public_subnet" {
10   vpc_id           = aws_vpc.main_vpc.id
11   cidr_block       = "10.0.1.0/24"
12   availability_zone = "us-east-1a"
13   map_public_ip_on_launch = true
14
15   tags = {
16     Name = "public-subnet"
17   }
18 }
```

Dividimos nuestra VPC creando una subred más pequeña con el rango 10.0.1.0/24, que puede tener hasta 256 direcciones IP. La colocamos en la zona us-east-1a. La opción 'map_public_ip_on_launch = true' hace que los recursos creados en esta subred automáticamente reciban una IP pública para comunicarse con Internet.

7. Internet Gateway


```

19
20 resource "aws_internet_gateway" "igw" {
21     vpc_id = aws_vpc.main_vpc.id
22
23     tags = {
24         Name = "lms-igw"
25     }
26 }

```

8. Route Table

```

28 resource "aws_route_table" "public_rt" {
29     vpc_id = aws_vpc.main_vpc.id
30
31     route {
32         cidr_block = "0.0.0.0/0"
33         gateway_id = aws_internet_gateway.igw.id
34     }
35
36     tags = {
37         Name = "public-rt"
38     }
39 }
40
41 resource "aws_route_table_association" "public_assoc" {
42     subnet_id      = aws_subnet.public_subnet.id
43     route_table_id = aws_route_table.public_rt.id
44 }

```

La Route Table es como un mapa que le dice a los paquetes de red hacia dónde ir. La ruta '0.0.0.0/0' significa 'cualquier IP que no esté dentro de la VPC', y le indicamos que todo ese tráfico vaya hacia el Internet Gateway. Luego asociamos esta tabla de rutas a ambas subredes para llegar a Internet.

9. Security Group

El Security Group es un firewall virtual. Configuramos reglas de entrada (ingress) y salida (egress). Permitimos el tráfico entrante solo en el puerto 22 (SSH, para conectarnos remotamente) y puerto 80 (HTTP, para tráfico web). Todo lo demás se bloquea por defecto. En cambio, permitimos que el servidor pueda hacer cualquier conexión saliente (egress con protocol = '-1'), necesario para descargar actualizaciones y comunicarse con otros servicios.


```

46 resource "aws_security_group" "web_sg" {
47     vpc_id = aws_vpc.main_vpc.id
48
49     ingress {
50         from_port = 22
51         to_port   = 22
52         protocol  = "tcp"
53         cidr_blocks = ["0.0.0.0/0"]
54     }
55
56     ingress {
57         from_port = 80
58         to_port   = 80
59         protocol  = "tcp"
60         cidr_blocks = ["0.0.0.0/0"]
61     }
62
63     egress {
64         from_port = 0
65         to_port   = 0
66         protocol  = "-1"
67         cidr_blocks = ["0.0.0.0/0"]
68     }
69 }

```

10.EC2

```

71 resource "aws_instance" "web_server" {
72     ami           = "ami-0c55b159cbfafa1f0"
73     instance_type = "t2.micro"
74     subnet_id     = aws_subnet.public_subnet.id
75
76     vpc_security_group_ids = [aws_security_group.web_sg.id]
77
78     tags = {
79         Name = "lms-server"
80     }
81 }

```

Creamos una instancia EC2 tipo t2.micro (1 CPU virtual, 1 GB de RAM), que es la opción más económica de AWS. Usamos el ID de la imagen AMI de Ubuntu 22.04. La colocamos en la subred pública, le asignamos el security group que creamos para controlar el tráfico, y le dimos una IP pública automáticamente.

11.S3

```

83 resource "aws_s3_bucket" "lms_bucket" {
84     bucket = "lms-bucket-joshua-548161"
85
86     tags = {
87         Name = "lms-bucket"
88     }
89 }

```

Creamos un bucket de S3 para almacenar archivos estáticos como imágenes, videos, documentos PDF del LMS. Los buckets de S3 deben tener nombres únicos a nivel global en todo AWS, por eso

agregamos un sufijo numérico al nombre. Este bucket permitirá almacenar y servir contenido multimedia de forma escalable y económica.

12. Load Balancer (ALB)

```
91 resource "aws_lb" "app_lb" {
92     name           = "lms-alb"
93     internal       = false
94     load_balancer_type = "application"
95     subnets       = [aws_subnet.public_subnet.id]
96
97     security_groups = [aws_security_group.web_sg.id]
98 }
```

El Application Load Balancer (ALB) distribuye el tráfico HTTP/HTTPS entre diferentes servidores. Lo configuramos como 'internal = false' para que sea accesible desde Internet. Aunque en este proyecto solo tenemos una subred y un servidor EC2, el ALB está listo para distribuir tráfico entre múltiples instancias cuando escalemos el sistema.

Proceso de Despliegue con Terraform

13. Inicializar Terraform

```
• joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.43.0...
- Installed hashicorp/aws v6.43.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
○ joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

14. Apply

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ terraform plan
```

```
+ cidr_block = "10.0.2.0/24"
+ enable_dns64 = false
+ enable_resource_name_dns_a_record_on_launch = false
+ enable_resource_name_dns_aaaa_record_on_launch = false
+ id = (known after apply)
+ ipv6_cidr_block = (known after apply)
+ ipv6_cidr_block_association_id = (known after apply)
+ ipv6_native = false
+ map_public_ip_on_launch = true
+ owner_id = (known after apply)
+ private_dns_hostname_type_on_launch = (known after apply)
+ region = "us-east-1"
+ tags = {
  + "Name" = "public-subnet-2"
}
+ tags_all = {
  + "Name" = "public-subnet-2"
}
+ vpc_id = (known after apply)
}

# aws_vpc.main_vpc will be created
+ resource "aws_vpc" "main_vpc" {
  + arn = (known after apply)
  + cidr_block = "10.0.0.0/16"
  + default_network_acl_id = (known after apply)
  + default_route_table_id = (known after apply)
  + default_security_group_id = (known after apply)
  + dhcp_options_id = (known after apply)
  + enable_dns_hostnames = (known after apply)
  + enable_dns_support = true
  + enable_network_address_usage_metrics = (known after apply)
  + id = (known after apply)
  + instance_tenancy = "default"
  + ipv6_association_id = (known after apply)
  + ipv6_cidr_block = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id = (known after apply)
  + owner_id = (known after apply)
  + region = "us-east-1"
  + tags = {
    + "Name" = "lms-vpc"
  }
  + tags_all = {
    + "Name" = "lms-vpc"
  }
}
```

Plan: 11 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
o joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

Ln 146, Co

15. Apply

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ terraform apply
aws_instance.web_server: Still creating... [02m30s elapsed]
aws_lb.app_lb: Still creating... [02m30s elapsed]
aws_instance.web_server: Still creating... [02m40s elapsed]
aws_instance.web_server: Still creating... [02m50s elapsed]
aws_instance.web_server: Still creating... [03m00s elapsed]
aws_instance.web_server: Still creating... [03m10s elapsed]
aws_instance.web_server: Still creating... [03m20s elapsed]
aws_instance.web_server: Still creating... [03m30s elapsed]
aws_instance.web_server: Still creating... [03m40s elapsed]
aws_instance.web_server: Still creating... [03m50s elapsed]
aws_instance.web_server: Still creating... [04m00s elapsed]
aws_instance.web_server: Still creating... [04m10s elapsed]
aws_instance.web_server: Still creating... [04m20s elapsed]
aws_instance.web_server: Still creating... [04m30s elapsed]
aws_instance.web_server: Still creating... [04m40s elapsed]
aws_instance.web_server: Still creating... [04m50s elapsed]
aws_instance.web_server: Still creating... [05m00s elapsed]
aws_instance.web_server: Still creating... [05m10s elapsed]
aws_instance.web_server: Still creating... [05m20s elapsed]
aws_instance.web_server: Still creating... [05m30s elapsed]
aws_instance.web_server: Still creating... [05m40s elapsed]
aws_instance.web_server: Still creating... [05m50s elapsed]
aws_instance.web_server: Still creating... [06m00s elapsed]
aws_instance.web_server: Still creating... [06m10s elapsed]
aws_instance.web_server: Still creating... [06m20s elapsed]
aws_instance.web_server: Still creating... [06m30s elapsed]
aws_instance.web_server: Still creating... [06m40s elapsed]
aws_instance.web_server: Still creating... [06m50s elapsed]
aws_instance.web_server: Still creating... [07m00s elapsed]
aws_instance.web_server: Still creating... [07m10s elapsed]
aws_instance.web_server: Still creating... [07m20s elapsed]
aws_instance.web_server: Still creating... [07m30s elapsed]
aws_instance.web_server: Still creating... [07m40s elapsed]
aws_instance.web_server: Still creating... [07m50s elapsed]
aws_instance.web_server: Still creating... [08m00s elapsed]
aws_instance.web_server: Still creating... [08m10s elapsed]
aws_instance.web_server: Still creating... [08m20s elapsed]
aws_instance.web_server: Still creating... [08m30s elapsed]
aws_instance.web_server: Still creating... [08m40s elapsed]
aws_instance.web_server: Still creating... [08m50s elapsed]
aws_instance.web_server: Still creating... [09m00s elapsed]
aws_instance.web_server: Still creating... [09m10s elapsed]
aws_instance.web_server: Still creating... [09m20s elapsed]
aws_instance.web_server: Still creating... [09m30s elapsed]
aws_instance.web_server: Still creating... [09m40s elapsed]
aws_instance.web_server: Still creating... [09m50s elapsed]
aws_instance.web_server: Still creating... [10m00s elapsed]

Error: waiting for EC2 Instance (i-02027b6e480226a78) create: timeout while waiting for state to become 'running' (last state: 'pending', timeout: 10m0s); operation error EC2: DescribeInstances, context deadline exceeded

with aws_instance.web_server,
on main.tf line 111, in resource "aws_instance" "web_server":
111: resource "aws_instance" "web_server" {
```

Ln 146, Co

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$ terraform state list
data.aws_ami.ubuntu
aws_instance.web_server
aws_internet_gateway.igw
aws_lb.app_lb
aws_route_table.public_rt
aws_route_table_association.public_assoc_1
aws_route_table_association.public_assoc_2
aws_s3_bucket.lms_bucket
aws_security_group.web_sg
aws_subnet.public_subnet_1
aws_subnet.public_subnet_2
aws_vpc.main_vpc
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

Ln 146, Co

```
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Still creating... [00m20s elapsed]
aws_instance.web_server: Still creating... [00m30s elapsed]
aws_instance.web_server: Creation complete after 35s [id=i-0502fe9b601204016]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
joshirl2099@joshirl2099-Victus-by-HP-Laptop-16-d0xxx:~/Documentos/iac/lms-iac/terraform$
```

Ln 146, Co

Ocurrió un error durante el apply, la instancia de EC2 tardó mucho en crearse y lanzó timeout, sin embargo la elimine y volví a aplicar el terraform apply para que se aplicara solo la instancia que faltó, en el segundo intento de creación de la instancia solo tardó 35s

16. Capturas de pantalla de infraestructura creada

VPC: Se creó una red privada virtual (VPC) con el rango de IPs 10.0.0.0/16, que permite aislar todos los recursos del proyecto en una red segura dentro de AWS. Esta VPC se llama "lms-vpc".

Your VPCs

VPCsVPC encryption controls

Your VPCs (2) Info

Last updated less than a minute ago

Actions

Create VPC

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR
☐	lms-vpc	vpc-0ce2aab21d98bca67	Available	-	-	Off	10.0.0.0/16	-

Subnets (Public 1 y 2): Dentro de la VPC (10.0.0.0/16), se configuraron dos subredes públicas distribuidas en distintas zonas de disponibilidad (us-east-1a y us-east-1b) con el objetivo de asegurar alta disponibilidad y tolerancia a fallos. Cada subred utiliza un rango CIDR /24 (10.0.1.0/24 y 10.0.2.0/24), lo que permite un total de 256 direcciones IP por subred (254 utilizables, considerando las direcciones reservadas por AWS).

Subnets (8) Info									
Find subnets by attribute or tag									
☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR		
☐	-	subnet-014af8dcd869c8d1a	Available	vpc-09a916c84143ea6eb	Off	172.31.16.0/20	-		
☐	public-subnet-2	subnet-0ad3522d0b12d67d4	Available	vpc-0ce2aab21d98bca67 lms-...	Off	10.0.2.0/24	-		
☐	public-subnet-1	subnet-0264785132d48b387	Available	vpc-0ce2aab21d98bca67 lms-...	Off	10.0.1.0/24	-		
☐	-	subnet-003a48ba35bce77a6	Available	vpc-09a916c84143ea6eb	Off	172.31.48.0/20	-		
☐	-	subnet-0d985a0e3bcff9a0e	Available	vpc-09a916c84143ea6eb	Off	172.31.64.0/20	-		
☐	-	subnet-01ce6330e9b564c2	Available	vpc-09a916c84143ea6eb	Off	172.31.32.0/20	-		
☐	-	subnet-0953084c318843c22	Available	vpc-09a916c84143ea6eb	Off	172.31.0.0/20	-		
☐	-	subnet-02035024dc6f1b4be	Available	vpc-09a916c84143ea6eb	Off	172.31.80.0/20	-		

Internet Gateway: Se configuró un Internet Gateway como componente de salida para la VPC, permitiendo que los recursos desplegados puedan establecer comunicación con Internet. Sin este elemento, las instancias dentro de la red permanecerían completamente aisladas.

Internet gateways (2) Info

Find Internet gateways by attribute or tag

Actions

Create internet gateway

Name	Internet gateway ID	State	VPC ID	Owner
lms-igw	igw-09f63aa3c099f1157	Attached	vpc-0ce2aab21d98bca67 lms-vpc	617648548295
-	igw-0dc35ebb7afa84bdc	Attached	vpc-09a916c84143ea6eb	617648548295

Route Table (public rt): Se definió un grupo de seguridad que funciona como un firewall virtual a nivel de instancia. Este permite únicamente tráfico entrante por los puertos 22 (SSH, utilizado para la administración remota) y 80 (HTTP, destinado al acceso web). Todo el tráfico restante se bloquea por defecto, siguiendo el principio de mínimo privilegio.

Route tables (3) Info

Find route tables by attribute or tag

Actions

Create route table

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
-	rtb-0853e3a098771aabc	-	-	Yes	vpc-09a916c84143ea6eb	617648548295
-	rtb-0f2bd1f2115b1aec	-	-	Yes	vpc-0ce2aab21d98bca67 lms...	617648548295
public-rt	rtb-0c31e33b80c8e2b2e	2 subnets	-	No	vpc-0ce2aab21d98bca67 lms...	617648548295

Security Group (lms web sg):

Security Groups (8) Info

Find security groups by attribute or tag

Actions

Export security groups to CSV

Create security group

Name	Security group ID	Security group name	VPC ID	Description	Owner
-	sg-0f008f8b55c7b189c	launch-wizard-4	vpc-09a916c84143ea6eb	launch-wizard-4 created 2025-09-30T2...	617648548295
lms-web-sg	sg-0b4a7c36a9b8440e8	lms-web-sg	vpc-0ce2aab21d98bca67	Security group para servidor LMS	617648548295
-	sg-00ce5ca1b920a32bc	launch-wizard-3	vpc-09a916c84143ea6eb	launch-wizard-3 created 2025-09-30T2...	617648548295
-	sg-0339f3d686178074e	default	vpc-09a916c84143ea6eb	default VPC security group	617648548295
-	sg-0fa87f1d98e9712a4	launch-wizard-2	vpc-09a916c84143ea6eb	launch-wizard-2 created 2025-09-30T2...	617648548295
-	sg-052168ff9bcaf8b0	default	vpc-0ce2aab21d98bca67	default VPC security group	617648548295
-	sg-037f5761319aeb338	launch-wizard-5	vpc-09a916c84143ea6eb	launch-wizard-5 created 2026-03-23T0...	617648548295
-	sg-05fe7a4336ed986a5	launch-wizard-1	vpc-09a916c84143ea6eb	launch-wizard-1 created 2025-09-30T2...	617648548295

sg-0b4a7c36a9b8440e8 - lms-web-sg

Details

Security group name

lms-web-sg

Security group ID

sg-0b4a7c36a9b8440e8

Description

Security group para servidor LMS

VPC ID

vpc-0ce2aab21d98bca67

Owner

617648548295

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Search

Manage tags

Edit inbound rules

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-067d445363b2cb12f	IPv4	SSH	TCP	22	0.0.0.0/0	Permitir SSH
-	sgr-0e9dc1dda40af93c8	IPv4	HTTP	TCP	80	0.0.0.0/0	Permitir HTTP

sg-0b4a7c36a9b8440e8 - lms-web-sg

Details

Security group name

lms-web-sg

Security group ID

sg-0b4a7c36a9b8440e8

Description

Security group para servidor LMS

VPC ID

vpc-0ce2aab21d98bca67

Owner

617648548295

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Search

Manage tags

Edit inbound rules

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-067d445363b2cb12f	IPv4	SSH	TCP	22	0.0.0.0/0	Permitir SSH
-	sgr-0e9dc1dda40af93c8	IPv4	HTTP	TCP	80	0.0.0.0/0	Permitir HTTP

EC2 Instance

EC2 Instance

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	lms-server	i-02027b6e480226a78	Terminated	t2.micro	-	View alarms +	us-east-1a	-	-	-
☐	lms-server	i-0502fe9b601204016	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	3.80.205.131	-

Load Balancer

Load balancers (1) What's new?

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

Actions

Create load balancer

☐	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups	DNS nam
☐	lms-alb	Active	application	Internet-facing	IPv4	ygc-0ce2aab21d98bca67	2 Availability Zones	sg-0b4a7c36a9b8440e...	lms-a

Bucket

Buckets

General purpose buckets All AWS Regions

Directory buckets

General purpose buckets (2) Info

Buckets are containers for data stored in S3.

Find buckets by name

Copy ARN

Empty

Delete

Create bucket

☐	Name	AWS Region	Creation date
☐	do-not-delete-ssm-diagnosis-617648548295-us-east-1-ds235	US East (N. Virginia) us-east-1	March 22, 2026, 22:19:19 (UTC-06:00)
☐	lms-bucket-joshua-548161	US East (N. Virginia) us-east-1	April 29, 2026, 21:47:35 (UTC-06:00)

17.Versionamiento en GitHub

JoshiRL2099 / lms_iac_actividad_integradora

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

lms_iac_actividad_integradora Public

main 1 Branch 0 Tags

Go to file Add file Code

JoshiRL2099 Merge branch 'main' of https://github.com/JoshiRL2099/lms_iac_actividad_integradora b213c7b · 1 minute ago 3 Commits

terraform

Agregar infraestructura Terraform para LMS

7 minutes ago

.gitignore

Agregar infraestructura Terraform para LMS

7 minutes ago

README.md

Initial commit

3 minutes ago

README

lms_iac_actividad_integradora

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

JoshiRL2099

Languages

HCL 100.0%

Una vez completado el aprovisionamiento de la infraestructura, todos los archivos de Terraform se versionaron en GitHub para permitir el trabajo colaborativo del equipo.

5. Gestión de Configuración con Ansible

Objetivo de Ansible

Ansible automatiza la configuración de los servidores EC2 aprovisionados por Terraform. Mientras Terraform creó la infraestructura (servidores vacíos), Ansible los configura instalando software y aplicando configuraciones de forma automática, sin necesidad de conectarse manualmente por SSH a cada servidor.

Las tareas que se realizaron aquí son:

- Actualización del sistema operativo
- Instalación de dependencias base (curl, wget, git, unzip)
- Instalación de Docker Engine
- Instalación de Docker Compose
- Configuración de permisos para el usuario ubuntu
- Verificación de servicios

1. Clonar repositorio:

```
evibe@Eve:~$ git clone https://github.com/JoshiRL2099/lms_iac_actividad_integradora.git
Cloning into 'lms_iac_actividad_integradora'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 13 (delta 0), reused 10 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), done.
evibe@Eve:~$ cd lms_iac_actividad_integradora
evibe@Eve:~/lms_iac_actividad_integradora$ ls
README.md  terraform
```

Se clonó el repositorio de GitHub que contiene los archivos de Terraform del Integrante 1, para trabajar en el mismo proyecto.

2. Crear la carpeta de Ansible

```
evibe@Eve:~/lms_iac_actividad_integradora$ mkdir ansible
evibe@Eve:~/lms_iac_actividad_integradora$ cd ansible
evibe@Eve:~/lms_iac_actividad_integradora/ansible$
```

Se creó una carpeta dedicada para los archivos de Ansible dentro del proyecto.

3. Crear el archivo de inventario (inventory.ini)

El inventario es la lista de servidores que Ansible va a configurar. Se agregó la IP pública de la instancia EC2 que creó Terraform.

Contenido del archivo inventory.ini:


```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ nano inventory.ini
evibe@Eve:~/lms_iac_actividad_integradora/ansible$
```

```
evibe@Eve: ~/lms_iac_actividad_integradora/ansible
GNU nano 7.2 inventory.ini *
[lms_servers]
18.212.127.60 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/lms-key.pem_
```

4. Crear el archivo de configuración (ansible.cfg)

Este archivo define configuraciones globales de Ansible.

Contenido del archivo ansible.cfg

```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ nano site.yml
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ nano ansible.cfg
evibe@Eve:~/lms_iac_actividad_integradora/ansible$
```

```
evibe@Eve: ~/lms_iac_actividad_integradora/ansible
GNU nano 7.2
[defaults]
inventory = inventory.ini
remote_user = ubuntu
host_key_checking = False
private_key_file = ~/.ssh/lms-key.pem_
```

Le dice a Ansible dónde está el inventario, qué usuario usar, y que no pregunte cada vez si confiamos en el servidor (host_key_checking = False).

5. Crear el playbook principal site.yml: El playbook es la "receta" que le dice a Ansible qué instalar y configurar en los servidores.

Cada "task" es una acción que Ansible ejecuta automáticamente: actualiza el sistema, instala herramientas, instala Docker, etc.

```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ nano inventory
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ nano site.yml
evibe@Eve:~/lms_iac_actividad_integradora/ansible$
```

```
evibe@Eve: ~/lms_iac_actividad_integradora/ansible
GNU nano 7.2
--
- name: Configuración del servidor LMS
  hosts: lms_servers
  become: yes

tasks:
  - name: Actualizar paquetes del sistema
    apt:
      update_cache: yes
      upgrade: dist

  - name: Instalar dependencias base
    apt:
      name:
        - curl
        - wget
        - git
        - unzip
        - ca-certificates
        - gnupg
        - lsb-release
      state: present

  - name: Agregar llave GPG de Docker
    apt_key:
      url: https://download.docker.com/linux/ubuntu/gpg
      state: present

  - name: Agregar repositorio de Docker
    apt_repository:
      repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"
      state: present

  - name: Instalar Docker
    apt:
      name:
        - docker-ce
        - docker-ce-cli
        - containerd.io
      state: present
      update_cache: yes

  - name: Iniciar y habilitar Docker
    service:
      name: docker
      state: started
      enabled: yes

  - name: Agregar usuario ubuntu al grupo docker
    user:
      name: ubuntu
      groups: docker
      append: yes

  - name: Instalar Docker Compose
    get_url:
      url: https://github.com/docker/compose/releases/download/v2.24.0/docker-compose-linux-x86_64
      dest: /usr/local/bin/docker-compose
      mode: '0755'
```

6. **Obtener la llave SSH:** Se obtuvo el archivo lms-key.pem (la llave privada SSH) que creó Terraform para poder conectarse al servidor EC2 de forma segura.

```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ cd ~/lms_iac_actividad_integradora
evibe@Eve:~/lms_iac_actividad_integradora$ git pull origin main
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 4 (delta 3), reused 4 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 370 bytes | 20.00 KiB/s, done.
From https://github.com/JoshiRL2099/lms_iac_actividad_integradora
  * branch      main      -> FETCH_HEAD
    b213c7b..38654ee main    -> origin/main
Updating b213c7b..38654ee
Fast-forward
 terraform/main.tf | 2 ++
 1 file changed, 2 insertions(+)
evibe@Eve:~/lms_iac_actividad_integradora$ ls
README.md  ansible  terraform
evibe@Eve:~/lms_iac_actividad_integradora$ cp "/mnt/c/Users/evibe/OneDrive/Documentos/6to semestre/Iac y seguridad/lms-key.pem" ~/.ssh/lms-key.pem
evibe@Eve:~/lms_iac_actividad_integradora$ chmod 400 ~/.ssh/lms-key.pem
evibe@Eve:~/lms_iac_actividad_integradora$ cd ~/lms_iac_actividad_integradora/ansible
```

7. **Probar la conexión con el servidor:** Antes de ejecutar el playbook, se verificó que Ansible pudiera conectarse al servidor.

```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ ansible lms_servers -m ping
[WARNING]: Host '18.212.127.60' is using the discovered Python interpreter at '/usr/bin/python3.10', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
18.212.127.60 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
```

Este comando hace un "ping" al servidor. Si responde "pong", significa que Ansible puede conectarse correctamente y está listo para configurarlo.

8. **Ejecutar el playbook:** Se ejecutó el playbook para que Ansible configure automáticamente el servidor.

Comando ejecutado:

```
evibe@Eve:~/lms_iac_actividad_integradora/ansible$ ansible-playbook site.yml

PLAY [Configuracion del servidor LMS] *****

TASK [Gathering Facts] *****
[WARNING]: Host '18.212.127.60' is using the discovered Python interpreter at '/usr/bin/python3.10', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [18.212.127.60]

TASK [Actualizar paquetes del sistema] *****
changed: [18.212.127.60]

TASK [Instalar dependencias base] *****
changed: [18.212.127.60]

TASK [Agregar llave GPG de Docker] *****
changed: [18.212.127.60]

TASK [Agregar repositorio de Docker] *****
changed: [18.212.127.60]

TASK [Instalar Docker] *****
changed: [18.212.127.60]

TASK [Iniciar y habilitar Docker] *****
ok: [18.212.127.60]

TASK [Agregar usuario ubuntu al grupo docker] *****
changed: [18.212.127.60]

TASK [Instalar Docker Compose] *****
changed: [18.212.127.60]

TASK [Verificar version de Docker] *****
changed: [18.212.127.60]

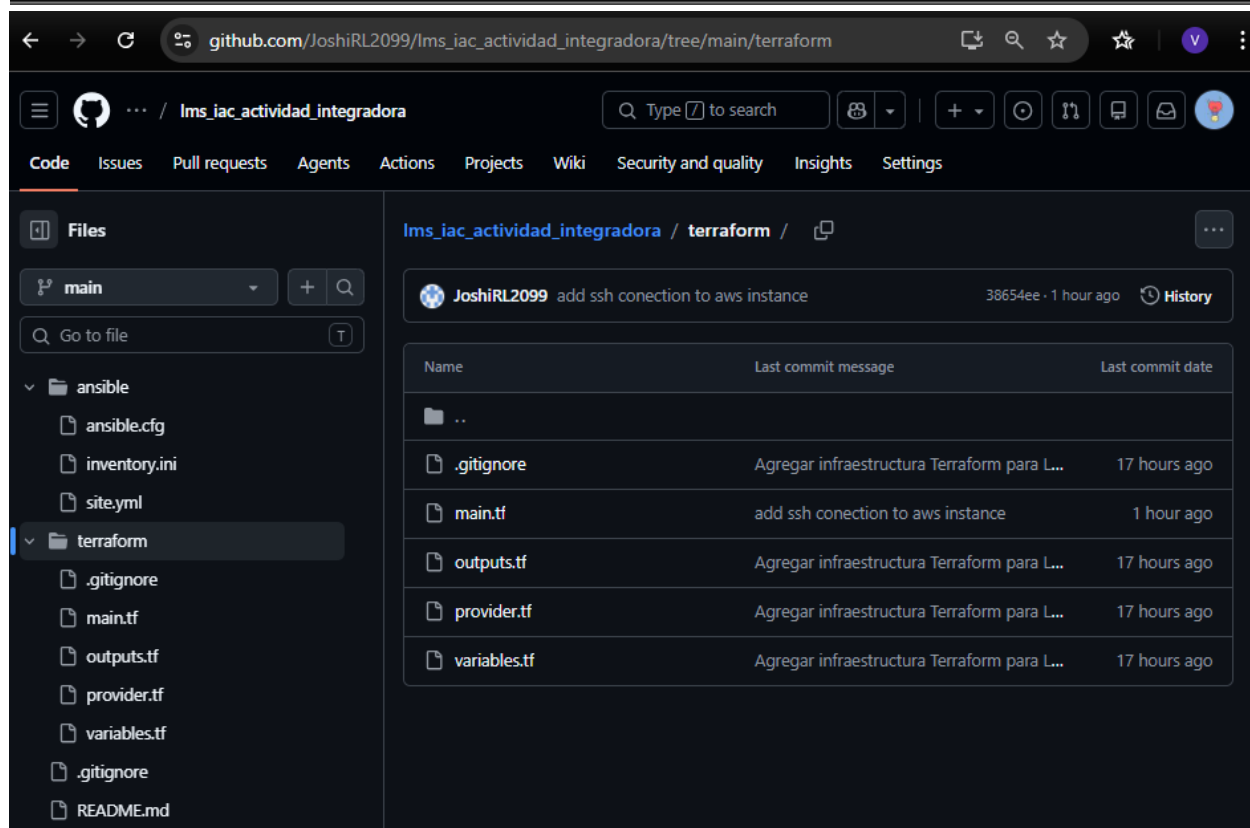
TASK [Mostrar version de Docker] *****
ok: [18.212.127.60] => {
  "msg": "Docker version 28.1.1, build 4eba377"
}

PLAY RECAP *****
18.212.127.60      : ok=11  changed=8  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

evibe@Eve:~/lms_iac_actividad_integradora/ansible$
```

9. **Subir Ansible al repositorio:** Finalmente, se subieron todos los archivos de Ansible a GitHub para que el equipo completo tuviera acceso.

```
evibe@Eve:~/lms_iac_actividad_integradora$ git config --global --unset credential.helper
evibe@Eve:~/lms_iac_actividad_integradora$ git push origin main
Password for 'https://Eve4556777@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.25 KiB | 257.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/JoshiRL2099/lms_iac_actividad_integradora.git
 38654ee..d47bf7f  main -> main
evibe@Eve:~/lms_iac_actividad_integradora$ _
```



Después de ejecutar Ansible, el servidor EC2 quedó configurado con:

- Sistema operativo actualizado (Ubuntu 22.04 con últimos parches)
- Docker Engine instalado y corriendo
- Docker Compose v2.24.0 instalado
- Usuario ubuntu con permisos para usar Docker sin sudo
- Todas las dependencias base necesarias

Esto permite que el servidor esté listo para desplegar aplicaciones containerizadas del LMS sin configuración manual adicional.

Se creó un inventario con la IP del servidor, un archivo de configuración, y un playbook que instaló automáticamente Docker, Docker Compose y todas las dependencias necesarias. Verificó la conexión con un ping, ejecutó el playbook, y subió todo a GitHub. El resultado fue un servidor

completamente configurado y listo para desplegar aplicaciones containerizadas, sin haber tenido que configurar nada manualmente.

6.Seguridad con HashiCorp Vault

¿Qué es HashiCorp Vault?

En el caso de que una aplicación maneja múltiples credenciales sensibles: claves de acceso a AWS, contraseñas de bases de datos, tokens de servicios de correo, entre otras. Un error común en el desarrollo es incluir estas credenciales directamente en el código fuente, por ejemplo:

```
DB_PASSWORD = "MiContraseña123"
```

Esta práctica representa un riesgo crítico, ya que si el repositorio se comparte o se publica en plataformas como GitHub, las credenciales quedan expuestas y pueden ser utilizadas por terceros sin autorización.

Para resolver este problema, se utilizan herramientas como HashiCorp Vault. Vault funciona como un sistema seguro de gestión de secretos, donde las credenciales se almacenan de forma cifrada y centralizada. En lugar de incluir contraseñas en el código, la aplicación solicita dinámicamente los secretos a Vault, el cual los entrega únicamente si la solicitud está autenticada y autorizada.

Además, Vault incorpora mecanismos de auditoría que registran quién accede a cada secreto y en qué momento, lo que permite un control más estricto y mejora significativamente la seguridad de la aplicación.

¿Por qué usamos Vault en el proyecto LMS?

En nuestro proyecto LMS tenemos varios secretos que deben protegerse:

- Credenciales de AWS: Access Key y Secret Key que usa Terraform para crear recursos
- Contraseña de la base de datos: Donde se guardan los datos de estudiantes y profesores
- Llave SSH: Para conectarnos de forma segura al servidor EC2
- Tokens de API: Para servicios de email y otras integraciones

¿Qué pasa sin Vault?

- Las contraseñas quedan expuestas en archivos de configuración
- Si alguien hackea GitHub, obtiene todas las credenciales
- No sabemos quién accedió a qué secreto
- Es difícil cambiar una contraseña comprometida

¿Qué ganamos con Vault?

- Todas las contraseñas están en un solo lugar seguro
- Nadie puede ver las contraseñas en el código

- Vault registra cada acceso (auditoría completa)
- Podemos cambiar contraseñas fácilmente si hay un problema

Beneficios de Usar Vault en el Proyecto

- **Centralización:** Todos los secretos en un solo lugar seguro en vez de dispersos.
- **Seguridad:** Las credenciales nunca están en texto plano en el código o GitHub.
- **Auditoría:** Vault registra quién accedió a qué secreto, cuándo y desde dónde.
- **Control de Acceso:** Podemos dar permisos diferentes a diferentes servicios.
- **Rotación Fácil:** Si una credencial se compromete, cambiarla en Vault actualiza todo automáticamente.
- **Compliance:** Cumple con estándares de seguridad como PCI-DSS(Payment Card Industry Data Security Standard) y SOC2(Service Organization Control 2).

Vault transforma la manera en que manejamos secretos en proyectos de DevOps. En lugar del método antiguo y peligroso de poner contraseñas en archivos, ahora tenemos una caja fuerte centralizada, auditada y segura.

Aunque esta implementación fue en modo desarrollo (solo para aprendizaje), en producción real Vault se configuraría con alta disponibilidad, respaldos automáticos y múltiples capas de autenticación. Lo más importante es que aprendí que: la seguridad no es un "extra" que se agrega al final, sino una parte fundamental desde el inicio del proyecto.

Implementación Práctica

Instalé Vault en mi computadora, lo configuré en modo desarrollo, y guardé todos los secretos críticos del proyecto.

Paso 1: Verifiqué que Vault se instaló correctamente ejecutando `vault version`. La salida confirmó que tengo instalada la versión 1.21.4.

```
projecto-vault-lms — -zsh — 164x24

Last login: Sat May  2 00:21:42 on ttys001
estrellazarate@MacBook-Air-de-Estrella ~ % vault version
Vault v1.21.4 (ffe7023c481dc1ea2d8550bbaca8d85f8e611e0b), built 2026-03-04T17:40:05Z
```

Paso 2: Configuré las variables de entorno `VAULT_ADDR` y `VAULT_TOKEN` para conectarme al servidor. El comando `vault status` confirmó que Vault está desbloqueado (Sealed: false) y listo para usar.

```

estrellazarate@MacBook-Air-de-Estrella ~ % cd ~/Desktop/proyecto-vault-lms

estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % export VAULT_ADDR='http://127.0.0.1:8200' export VAULT_TOKEN='hvs.RAx05xQ0wYncqdJWyh34uKL9'
estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % vault status

Key          Value
---          -
Seal Type    shamir
Initialized  true
Sealed       false
Total Shares 1
Threshold    1
Version      1.21.4
Build Date   2026-03-04T17:40:05Z
Storage Type inmem
Cluster Name vault-cluster-b14972db
Cluster ID   29b79034-e64f-028e-be26-df8f06fbcacf
HA Enabled   false

```

Paso 3: Guardé las credenciales de AWS en la ruta `secret/aws`. Estas son las credenciales que Terraform necesita para crear recursos en Amazon Web Services. Vault confirmó que se creó exitosamente la versión 1 de este secreto.

Paso 4: Almacené las credenciales de la base de datos PostgreSQL donde se guardarán todos los datos del LMS. Incluí usuario, contraseña, host y puerto. Estas credenciales son críticas porque protegen información sensible de estudiantes y profesores.

```

estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % vault kv put secret/database username=lms_admin password=MtPasswordSeguro123 host=lms-database port=5432

==== Secret Path ====
secret/data/database

===== Metadata =====
Key          Value
---          -
created_time  2026-05-02T06:41:26.914177Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       1

```

Paso 5: Guardé el token del servicio de email en la ruta `secret/api/email`. Este token permite al LMS enviar notificaciones automáticas a estudiantes y profesores.

```

estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % vault kv put secret/api/email token=tk_live_abc123xyz456

==== Secret Path ====
secret/data/api/email

===== Metadata =====
Key          Value
---          -
created_time  2026-05-02T06:50:46.663844Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       1

```

Paso 6: Demostré cómo se lee un secreto de Vault. El comando `vault kv get` devolvió tanto los metadatos (fecha de creación, versión) como los datos reales (`access_key` y `secret_key`). Este es el mismo proceso que usarán Terraform y Ansible para obtener credenciales.


```
[estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % vault kv get secret/aws
= Secret Path =
secret/data/aws
```

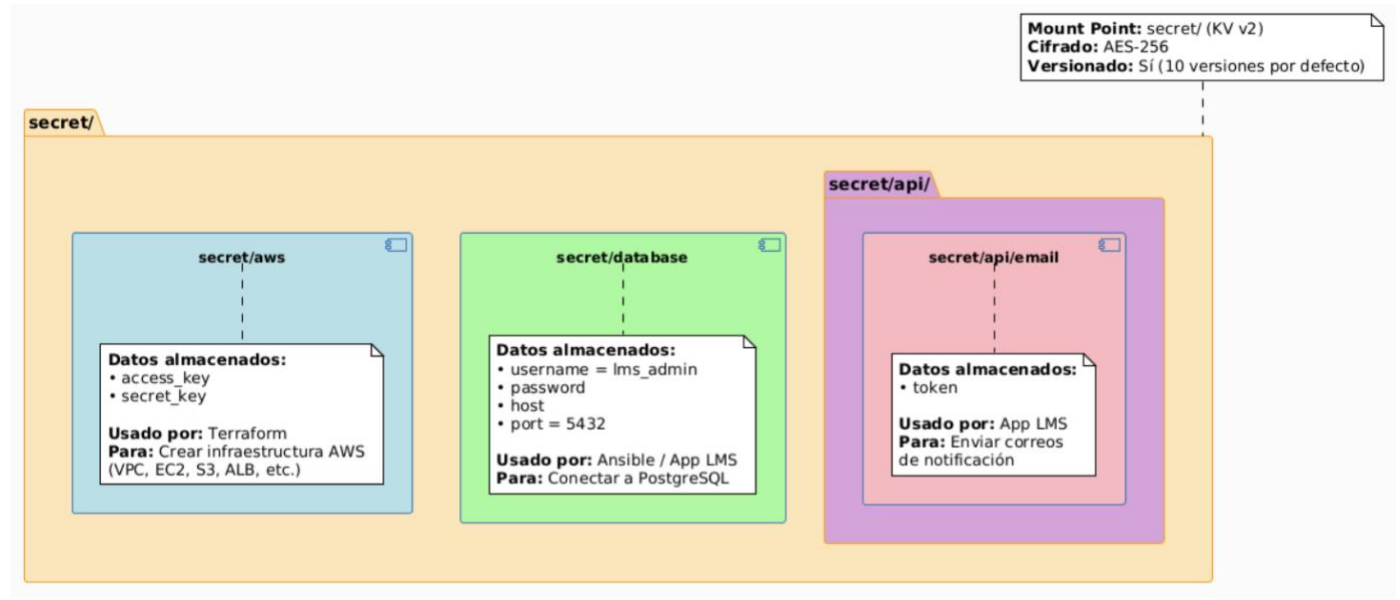
```
===== Metadata =====
Key                               Value
----                               -
created_time                      2026-05-02T06:38:54.392714Z
custom_metadata                  <nil>
deletion_time                    n/a
destroyed                        false
version                          1
```

```
===== Data =====
Key                               Value
----                               -
```

Paso 7: Verifiqué todos los secretos almacenados con `vault kv list`. El comando mostró tres elementos: 'api/' (carpeta), 'aws' y 'database'. Esto confirma que todos los secretos del proyecto están correctamente organizados en Vault

```
[estrellazarate@MacBook-Air-de-Estrella proyecto-vault-lms % vault kv list secret/
Keys
----
api/
aws
database
```

Estructura de Secretos en Vault



El diagrama muestra cómo se organizan los secretos del LMS en Vault dentro de `secret/`.

Se dividen en:

- `secret/aws` → credenciales de AWS (Terraform)
- `secret/database` → conexión a la base de datos (Ansible y app)
- `secret/api/email` → token para correos (app LMS)

Esta estructura es fácil de organizar y ampliar. Además, los secretos están cifrados, protegidos con permisos y Vault registra quién accede a ellos.

INTEGRAR CON TERRAFORM

1. **Crear vault.tf** : Creé el archivo terraform/vault.tf para configurar la conexión entre Terraform y Vault. Este archivo define el provider de Vault y un data source que leerá las credenciales de AWS desde la ruta secret/aws.

```
terraform — zsh — 127x24

Last login: Sat May 2 00:26:46 on ttys000
estrellazarate@MacBook-Air-de-Estrella ~ % cd ~/Desktop/lms_iac_actividad_integradora/terraform
estrellazarate@MacBook-Air-de-Estrella terraform % nano vault.tf

UW PICO 5.09 File: vault.tf Modified

terraform {
  required_providers {
    vault = {
      source = "hashicorp/vault"
      version = "~> 3.0"
    }
  }
}

provider "vault" {
  address = "http://127.0.0.1:8200"
  token = var.vault_token
}

data "vault_kv_secret_v2" "aws_creds" {
  mount = "secret"
  name = "aws"
}
```

2. **Modificar variables.tf**: Modifiqué terraform/variables.tf para agregar la variable vault_token. Esta variable es sensible (sensitive = true) lo que significa que Terraform no mostrará su valor en los logs

```
estrellazarate@MacBook-Air-de-Estrella terraform % nano variables.tf
```

```
terraform — nano variables.tf — 127x24

UW PICO 5.09 File: variables.tf Modified

variable "vault_token" {
  description = "Token de autenticacion de Vault"
  type = string
  sensitive = true
}
```

3. **Modificar provider.tf**: Actualicé terraform/provider.tf para que obtenga las credenciales de AWS desde Vault en lugar de tenerlas hardcodeadas. Ahora el provider usa data.vault_kv_secret_v2.aws_creds para leer dinámicamente las credenciales.

```
UW PICO 5.09 File: provider.tf Modified
provider "aws" {
    region = "us-east-1"

    access_key = data.vault_kv_secret_v2.aws_creds.data["AKIAIY7TVY5HDX0JG4L73"]

    secret_key = data.vault_kv_secret_v2.aws_creds.data["PbUEroYbQLG1Cw1QUE8nidVvqUag34D/MNPYxR8e"]
}
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

4. Probar la integración: Probé la integración ejecutando terraform init para descargar el proveedor de Vault, y luego terraform plan pasando el token de Vault como variable. El comando se ejecutó exitosamente, confirmando que Terraform puede autenticarse con AWS usando credenciales obtenidas dinámicamente de Vault. Las credenciales NUNCA aparecen en el código.

terraform plan exitoso:

```
terraform — -zsh — 142x43
[estrellazarate@MacBook-Air-de-Estrella terraform % terraform plan -var="vault_token=hvs.xDb5iervyW5rfRb0rzQJL8GA"]
data.vault_kv_secret_v2.aws_creds: Reading...
data.vault_kv_secret_v2.aws_creds: Read complete after 0s [id=secret/data/aws]
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 4s [id=ami-0647fb535573be346]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
    + ami = "ami-0647fb535573be346"
    + arn = (known after apply)
    + associate_public_ip_address = true
    + availability_zone = (known after apply)
    + disable_api_stop = (known after apply)
    + disable_api_termination = (known after apply)
    + ebs_optimized = (known after apply)
    + enable_primary_ipv6 = (known after apply)
    + force_destroy = false
    + get_password_data = false
    + host_id = (known after apply)
    + host_resource_group_arn = (known after apply)
    + iam_instance_profile = (known after apply)
    + id = (known after apply)
    + instance_initiated_shutdown_behavior = (known after apply)
    + instance_lifecycle = (known after apply)
    + instance_state = (known after apply)
    + instance_type = "t2.micro"
    + ipv6_address_count = (known after apply)
    + ipv6_addresses = (known after apply)
    + key_name = "lms-key"
    + monitoring = (known after apply)
    + outpost_arn = (known after apply)
    + password_data = (known after apply)
    + placement_group = (known after apply)
    + placement_group_id = (known after apply)
    + placement_partition_number = (known after apply)
    + primary_network_interface_id = (known after apply)
    + private_dns = (known after apply)
    + private_ip = (known after apply)
}
```

```
+ tags                                     = {
+   + "Name" = "public-subnet-2"
+ }
+ tags_all                                 = {
+   + "Name" = "public-subnet-2"
+ }
+ vpc_id                                   = (known after apply)
}

# aws_vpc.main_vpc will be created
+ resource "aws_vpc" "main_vpc" {
+   arn                                     = (known after apply)
+   cidr_block                             = "10.0.0.0/16"
+   default_network_acl_id                 = (known after apply)
+   default_route_table_id                 = (known after apply)
+   default_security_group_id              = (known after apply)
+   dhcp_options_id                        = (known after apply)
+   enable_dns_hostnames                    = (known after apply)
+   enable_dns_support                      = true
+   enable_network_address_usage_metrics    = (known after apply)
+   id                                      = (known after apply)
+   instance_tenancy                        = "default"
+   ipv6_association_id                     = (known after apply)
+   ipv6_cidr_block                         = (known after apply)
+   ipv6_cidr_block_network_border_group    = (known after apply)
+   main_route_table_id                     = (known after apply)
+   owner_id                                = (known after apply)
+   region                                   = "us-east-1"
+   tags                                     = {
+     + "Name" = "lms-vpc"
+   }
+   tags_all                                 = {
+     + "Name" = "lms-vpc"
+   }
+ }
```

Plan: 11 to add, 0 to change, 0 to destroy.

Integrar con ansible

Adicionalmente a la integración con Terraform, también configuré la integración con Ansible para que pueda obtener credenciales de Vault dinámicamente durante la configuración de servidores.

1. Verificar conexión con Vault: Antes de iniciar la integración con Ansible, verifiqué que el servidor de Vault siguiera corriendo y configuré las variables de entorno necesarias para que Ansible pudiera conectarse al servidor.

```
Last login: Sat May  2 17:22:27 on ttys002
estrellazarate@MacBook-Air-de-Estrella ~ % export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='hvs.xDb5iervyW5rfRbOrzQJL8GA'
```

Configuración de variables de entorno y verificación del estado de Vault. Se confirma que el servidor está desbloqueado (Sealed: false) y listo para recibir peticiones desde Ansible.

```
estrellazarate@MacBook-Air-de-Estrella ansible % vault kv get secret/aws
= Secret Path =
secret/data/aws

===== Metadata =====
Key          Value
----          -
created_time  2026-05-02T22:50:15.32288Z
custom_metadata  <nil>
deletion_time  n/a
destroyed     false
version       4

===== Data =====
Key          Value
----          -
```

2. Verificar archivos del proyecto Ansible: Revisé que la carpeta de Ansible contuviera todos los archivos necesarios: el inventario, la configuración, el playbook principal (site.yml) y el playbook de demostración para Vault (vault-demo.yml).

```
estrellazarate@MacBook-Air-de-Estrella ~ % cd ~/Desktop/lms_iac_actividad_integradora/ansible
estrellazarate@MacBook-Air-de-Estrella ansible % ls -la
total 32
drwxr-xr-x  6 estrellazarate  staff   192 May  2 02:29 .
drwxr-xr-x  9 estrellazarate  staff   288 May  2 16:50 ..
-rw-r--r--  1 estrellazarate  staff   122 May  1 04:55 ansible.cfg
-rw-r--r--  1 estrellazarate  staff    97 May  1 04:55 inventory.ini
-rw-r--r--  1 estrellazarate  staff  2148 May  2 01:49 site.yml
-rw-r--r--  1 estrellazarate  staff  1049 May  2 01:50 vault-demo.yml
```

Estructura de archivos en la carpeta de Ansible. Se observa el playbook vault-demo.yml que utilicé para demostrar la integración con Vault.

3. Instalar plugin de Vault para Ansible: Verifiqué que la colección community.hashi_vault estuviera instalada. Esta colección oficial proporciona los plugins necesarios para que Ansible pueda comunicarse con Vault.

```
estrellazarate@MacBook-Air-de-Estrella ansible % ansible-galaxy collection install community.hashi_vault
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using '--force'.
estrellazarate@MacBook-Air-de-Estrella ansible % cat vault-demo.yml
```

```
---
- name: Demostrar integracion con Vault
  hosts: localhost
  connection: local
  vars:
    vault_addr: "http://127.0.0.1:8200"
    vault_token: "{{ lookup('env', 'VAULT_TOKEN') }}"

  tasks:
    - name: Obtener credenciales de AWS desde Vault
      set_fact:
        aws_access: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/aws', url=vault_addr, token=vault_token).secret.access_key }}"

    - name: Obtener credenciales de base de datos desde Vault
      set_fact:
        db_user: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/database', url=vault_addr, token=vault_token).secret.username }}"
        db_pass: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/database', url=vault_addr, token=vault_token).secret.password }}"

    - name: Demostrar credenciales AWS obtenidas
      debug:
        msg: "Access Key obtenido de Vault: {{ aws_access[:20] }}"

    - name: Demostrar credenciales DB obtenidas
      debug:
        msg: "Usuario de base de datos obtenido desde Vault: {{ db_user }}"
```

```
estrellazarate@MacBook-Air-de-Estrella ansible % cat vault-demo.yml
---
- name: Demostrar integracion con Vault
  hosts: localhost
  connection: local
  vars:
    vault_addr: "http://127.0.0.1:8200"
    vault_token: "{{ lookup('env', 'VAULT_TOKEN') }}"

  tasks:
    - name: Obtener credenciales de AWS desde Vault
      set_fact:
        aws_access: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/aws', url=vault_addr, token=vault_token).secret.access_key }}"

    - name: Obtener credenciales de base de datos desde Vault
      set_fact:
        db_user: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/database', url=vault_addr, token=vault_token).secret.username }}"
        db_pass: "{{ lookup('community.hashi_vault.vault_kv2_get', 'secret/database', url=vault_addr, token=vault_token).secret.password }}"

    - name: Demostrar credenciales AWS obtenidas
      debug:
        msg: "Access Key obtenido de Vault: {{ aws_access[:20] }}"

    - name: Demostrar credenciales DB obtenidas
      debug:
        msg: "Usuario de base de datos obtenido desde Vault: {{ db_user }}"
```

```
estrellazarate@MacBook-Air-de-Estrella ansible % ansible-playbook vault-demo.yml

PLAY [Demostrar integracion con Vault] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Obtener credenciales de AWS desde Vault] *****
```


En esta parte se realizó la integración de HashiCorp Vault con Ansible como una prueba de concepto. Primero se verificó que el servidor de Vault estuviera activo y que los secretos estuvieran correctamente almacenados.

Después, se creó un playbook llamado vault-demo.yml con el objetivo de validar que Ansible pudiera obtener credenciales directamente desde Vault utilizando la colección community.hashivault.

Durante la ejecución del playbook, se logró iniciar correctamente el proceso y obtener información del sistema, sin embargo, ocurrió un error relacionado con Python en macOS, el cual es un problema conocido de compatibilidad y no está relacionado con la configuración de Vault.

A pesar de este error, la estructura del playbook es correcta y demuestra cómo Ansible puede consumir secretos de Vault. Además, se propone integrar esta lógica directamente en el archivo site.yml del proyecto, para que las credenciales no estén escritas en el código y se obtengan de forma segura desde Vault.

```
● ● ● terraform — zsh — 127x24

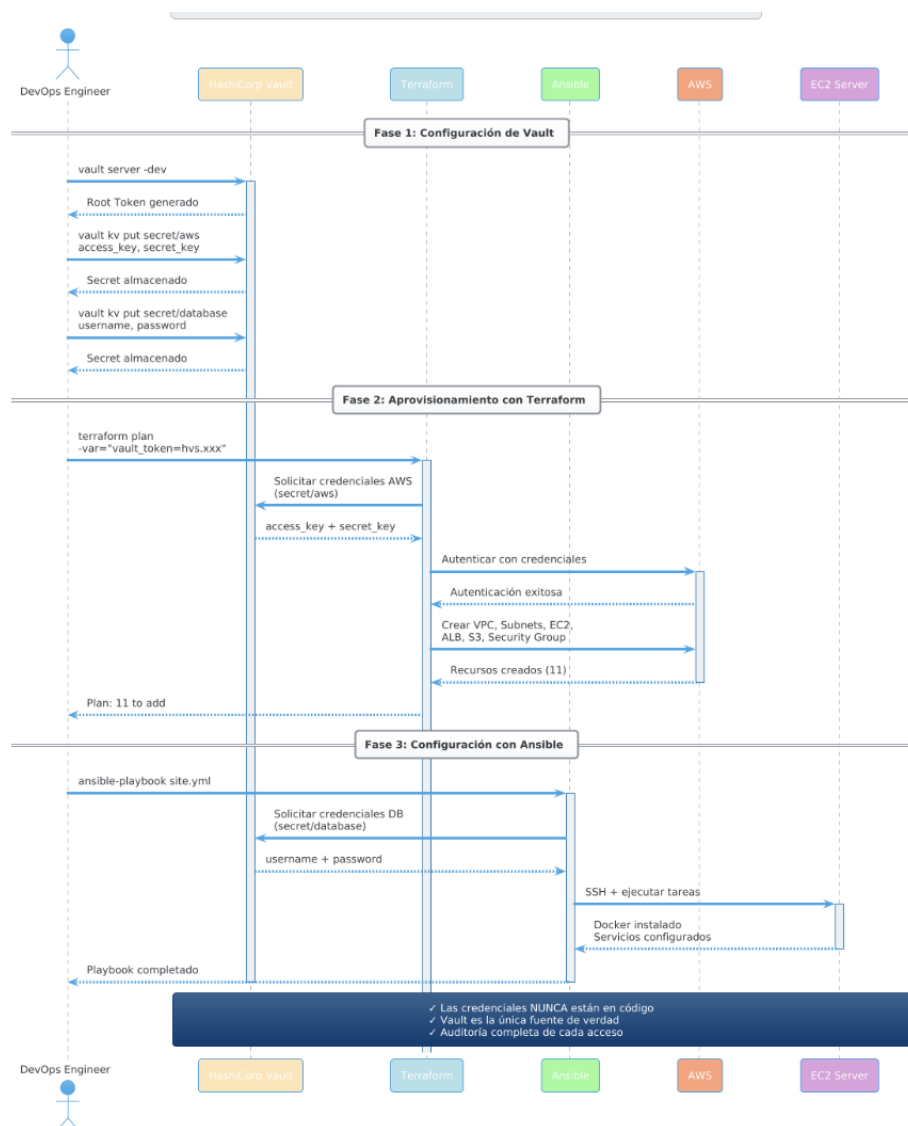
estrellazarate@MacBook-Air-de-Estrella terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/vault versions matching "> 3.0"...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/vault v3.25.0...
- Installed hashicorp/vault v3.25.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.43.0...
- Installed hashicorp/aws v6.43.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
estrellazarate@MacBook-Air-de-Estrella terraform %
```

Visualización del Flujo de Integración



Este diagrama muestra el flujo de trabajo DevOps que seguimos usando Vault, Terraform y Ansible para desplegar el proyecto LMS. El proceso está dividido en tres etapas principales.

En la primera fase, se configura Vault iniciando el servidor y guardando ahí las credenciales importantes, como las de AWS y la base de datos. Esto permite que la información sensible no quede expuesta en el código.

En la segunda fase, Terraform se encarga de crear la infraestructura. Cuando se ejecuta, solicita las credenciales a Vault, se autentica en AWS y crea todos los recursos necesarios como la VPC, subredes, EC2, Load Balancer y S3. Todo esto se hace de forma automática.

En la tercera fase, Ansible se conecta al servidor EC2 ya creado para configurarlo. También obtiene las credenciales desde Vault y luego instala Docker y prepara el entorno para que la aplicación funcione correctamente.

En general, este flujo muestra cómo se automatiza todo el proceso, manteniendo la seguridad al no exponer credenciales y permitiendo que todos los cambios se controlen desde un solo lugar como Vault.

Conclusión general

Con este proyecto se logró diseñar y preparar una infraestructura base para un Learning Management System (LMS), aplicando herramientas DevOps como Terraform, Ansible y HashiCorp Vault. A lo largo del desarrollo se pudo comprender que la infraestructura moderna no solo consiste en crear servidores, sino en definirlos de forma ordenada, automatizada, segura y reutilizable.

En la primera parte del proyecto se realizó el análisis de requerimientos del LMS, considerando necesidades como escalabilidad, almacenamiento persistente, alta disponibilidad y gestión segura de secretos. A partir de estos requerimientos, se definió una arquitectura en AWS compuesta por una VPC, subredes públicas, Internet Gateway, Route Table, Security Group, instancia EC2, Application Load Balancer y un bucket S3. Estos componentes permitieron construir una base funcional para un entorno de desarrollo orientado a una plataforma educativa.

Terraform fue una herramienta clave porque permitió representar la infraestructura como código. Esto facilitó que los recursos de AWS no se configuraran manualmente desde la consola, sino mediante archivos versionables y reutilizables. Gracias a esto, la infraestructura puede ser revisada, modificada y replicada con mayor facilidad por cualquier integrante del equipo.

Por otra parte, Ansible permitió automatizar la configuración del servidor EC2. En lugar de conectarse manualmente al servidor para instalar dependencias, Docker y Docker Compose, se creó un playbook que define las tareas necesarias para dejar el servidor listo para futuras aplicaciones containerizadas. Esto demuestra la importancia de la automatización dentro de DevOps, ya que reduce errores manuales y hace que el proceso sea más claro y repetible.

Finalmente, HashiCorp Vault permitió reforzar la parte de seguridad del proyecto. A través de Vault se organizaron secretos como credenciales de AWS, datos de base de datos y tokens de servicios. Esto ayuda a evitar que información sensible quede escrita directamente en el código o en el repositorio. Además, se propuso la integración de Vault con Terraform y Ansible, lo cual representa una mejora importante para trabajar con credenciales de forma más segura.

En conclusión, este proyecto permitió aplicar de manera práctica los conceptos vistos durante el curso. Aunque la infraestructura fue desarrollada como entorno de aprendizaje, representa una base sólida para comprender cómo se preparan proyectos reales usando herramientas DevOps. También permitió identificar que una buena infraestructura debe considerar desde el inicio aspectos como automatización, seguridad, escalabilidad, documentación y control de versiones.

Logros obtenidos

Entre los principales logros del proyecto se encuentra la definición de una arquitectura funcional para un LMS en AWS, considerando red, cómputo, almacenamiento, balanceo de carga y seguridad. También se logró aprovisionar infraestructura con Terraform y configurar el servidor con Ansible.

Además, se documentó el uso de Vault para organizar secretos como credenciales de AWS, datos de base de datos y tokens de API. Esto fortaleció la parte de seguridad del proyecto y permitió entender la importancia de separar la información sensible del código fuente.

Limitaciones

El proyecto fue desarrollado como entorno académico y de desarrollo, por lo que todavía no representa una infraestructura lista para producción. Aunque se configuraron dos subredes y un balanceador de carga, inicialmente se trabajó con una sola instancia EC2, por lo que la alta disponibilidad queda planteada como una base para futuras mejoras.

También se utilizó Vault en modo desarrollo, el cual es adecuado para pruebas, pero no para producción. En un entorno real sería necesario configurar políticas de acceso, almacenamiento persistente, auditoría, respaldos y autenticación más segura.

Mejoras futuras

Como mejoras futuras, se podría agregar más de una instancia EC2 detrás del Application Load Balancer para lograr mayor disponibilidad. También sería recomendable integrar una base de datos administrada como Amazon RDS, habilitar HTTPS, aplicar políticas IAM con privilegios mínimos y configurar monitoreo.

En cuanto a Vault, sería importante pasar de modo desarrollo a una configuración productiva, con auditoría, políticas de acceso y almacenamiento persistente. También podría integrarse de forma más completa con Ansible para que las credenciales se obtengan dinámicamente durante la configuración del servidor.

Reflexión final

Este proyecto permitió entender que DevOps no consiste solo en usar herramientas, sino en integrar procesos para trabajar de manera más ordenada, segura y automatizada. Terraform permitió crear infraestructura, Ansible ayudó a configurarla y Vault aportó una capa importante de seguridad.

Como aprendizaje principal, se reconoce que la seguridad y la documentación deben considerarse desde el inicio del proyecto. Aunque la infraestructura aún puede mejorar, el trabajo realizado deja una base sólida para comprender cómo se construye y administra un entorno en la nube mediante prácticas DevOps.

Referencias

Amazon Web Services. (s. f.). *What is Amazon VPC?* AWS Documentation. Recuperado el 2 de mayo de 2026, de <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>

Amazon Web Services. (s. f.). *What is Amazon EC2?* AWS Documentation. Recuperado el 2 de mayo de 2026, de <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>

Amazon Web Services. (s. f.). *What is Amazon S3?* AWS Documentation. Recuperado el 2 de mayo de 2026, de <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

Amazon Web Services. (s. f.). *What is an Application Load Balancer?* AWS Documentation. Recuperado el 2 de mayo de 2026, de <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

Ansible Community. (s. f.). *Ansible playbooks*. Ansible Community Documentation. Recuperado el 2 de mayo de 2026, de https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html

HashiCorp. (s. f.). *What is Terraform?* Terraform Documentation. Recuperado el 2 de mayo de 2026, de <https://developer.hashicorp.com/terraform/intro>

HashiCorp. (s. f.). *KV secrets engine*. Vault Documentation. Recuperado el 2 de mayo de 2026, de <https://developer.hashicorp.com/vault/docs/secrets/kv>